

struct y match

Unidad 0: Herramientas y preliminares

Sesión 3: Datos que vienen en variantes

Eduardo Acuña Yeomans

Semestre 2026-2

Lenguajes de Programación · LCC, Universidad de Sonora

Dónde estamos

Lo anterior

- Recursión: caso base y caso recursivo.
- Listas: `cons`, `car`, `cdr`, `null?`.
- Una función puede recibir otra función.

Esta sesión

- Cómo se guarda un dato que viene en **variantes**.
- `struct` para definirlo.
- `match` para procesarlo.

Queremos el área de una figura

Un círculo se describe con un radio. Un rectángulo, con base y altura.
Son la misma clase de cosa y no se guardan igual.

Queremos el área de una figura

Un círculo se describe con un radio. Un rectángulo, con base y altura.
Son la misma clase de cosa y no se guardan igual.

Con lo que sabemos hasta ahora, listas:

```
(define c (list 'circulo 3))  
(define r (list 'rectangulo 2 5))
```

Queremos el área de una figura

Un círculo se describe con un radio. Un rectángulo, con base y altura.
Son la misma clase de cosa y no se guardan igual.

Con lo que sabemos hasta ahora, listas:

```
(define c (list 'circulo 3))  
(define r (list 'rectangulo 2 5))
```

- ¿Cuál de los dos números de `r` es la base? Hay que acordarse.
- `(car (cdr r))` no dice nada sobre lo que saca.
- Nada impide escribir `(list 'circulo)` y quedarse sin radio.

struct

```
(struct circulo (radio) #:transparent)  
(struct rectangulo (base altura) #:transparent)
```

Cada línea define un tipo de dato con campos con nombre, y te da tres cosas:

- un **constructor**: `circulo`, `rectangulo`
- un **selector** por campo: `circulo-radio`, `rectangulo-base`, `rectangulo-altura`
- un **predicado**: `circulo?`, `rectangulo?`

`#:transparent` es para que el REPL nos deje ver lo que hay adentro.
Sin eso imprime `#<circulo>` y ya.

En el REPL

```
> (define c (circulo 3))  
> c  
(circulo 3)  
> (circulo-radio c)  
3  
> (circulo? c)  
#t  
> (rectangulo? c)  
#f  
> (circulo 3 4)  
circulo: arity mismatch
```

El último error es una buena noticia: un círculo tiene exactamente un campo, y Racket lo sabe.

Qué ganamos

Con listas

```
(list 'rectangulo 2 5)
```

```
(car (cdr r))
```

“¿era base o altura?”

un campo de menos: pasa

Con struct

```
(rectangulo 2 5)
```

```
(rectangulo-base r)
```

el nombre lo dice

un campo de menos: error

Un **struct** no guarda más información que la lista.
Guarda la misma, pero con nombre y con forma fija.

Ahora, el área

Una sola función que sirva para las dos variantes:

```
(define (area f)
  (if (circulo? f)
      (* 3.1416 (circulo-radio f) (circulo-radio f))
      (if (rectangulo? f)
          (* (rectangulo-base f) (rectangulo-altura f))
          (error "no es una figura"))))
```

Ahora, el área

Una sola función que sirva para las dos variantes:

```
(define (area f)
  (if (circulo? f)
      (* 3.1416 (circulo-radio f) (circulo-radio f))
      (if (rectangulo? f)
          (* (rectangulo-base f) (rectangulo-altura f))
          (error "no es una figura"))))
```

- Funciona.
- Pregunta de qué variante es, y luego saca los campos uno por uno.
- Con cuatro variantes esto es ilegible.

Lo mismo, con match

```
(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]))
```

- Un caso por variante, y ya.
- `match` hace las dos cosas de golpe: reconoce la variante y le pone nombre a los campos.
- No aparece ningún selector. No hace falta.

En `#lang racket` ya viene. En `#lang racket/base` hay que pedir
(`require racket/match`).

Cómo se lee un caso

```
[(circulo r) (* 3.1416 r r)]
```

- A la izquierda, el **patrón**. A la derecha, qué hacer si el patrón aplica.
- `(circulo r)` *parece* una llamada al constructor, pero aquí no construye nada: **desarma**.
- `r` no es una variable que ya exista. Es un nombre nuevo que **match** le pone al campo, y que solo vive dentro de ese caso.

Cómo se lee un caso

```
[(circulo r) (* 3.1416 r r)]
```

- A la izquierda, el **patrón**. A la derecha, qué hacer si el patrón aplica.
- `(circulo r)` *parece* una llamada al constructor, pero aquí no construye nada: **desarma**.
- `r` no es una variable que ya exista. Es un nombre nuevo que `match` le pone al campo, y que solo vive dentro de ese caso.

Si un campo no te interesa, se ignora con `_`: `[(rectangulo b _) b]`.

Un caso por variante

El dato tiene...	...la función tiene
la lista vacía <i>o</i> un <code>cons</code>	dos casos
un círculo <i>o</i> un rectángulo	dos casos
tres variantes	tres casos

La forma del dato dicta la forma de la función.

Con listas, con figuras. **Y en la próxima sesión, con programas.**

Ejercicio: agrega el cuadrado

Un cuadrado se describe con un lado. Define el **struct** y extiende **area**.

Ejercicio: agrega el cuadrado

Un cuadrado se describe con un lado. Define el `struct` y extiende `area`.

```
(struct cuadrado (lado) #:transparent)
```

```
(define (area f)  
  (match f  
    [(circulo r) (* 3.1416 r r)]  
    [(rectangulo b h) (* b h)]  
    [(cuadrado l) (* l l)]))
```


Ejercicio: agrega el cuadrado

Un cuadrado se describe con un lado. Define el `struct` y extiende `area`.

```
(struct cuadrado (lado) #:transparent)
```

```
(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]
    [(cuadrado l) (* l l)]))
```

Una pregunta para dejarte quemando cinta: ¿hacía falta una variante nueva, o un cuadrado es un rectángulo con base igual a la altura?

Si te falta un caso

```
> (area (cuadrado 4))  
match: no matching clause for (cuadrado 4)
```

- Eso es lo que pasa si defines el **struct** y olvidas el caso en **match**.
- El error sale **cuando corres**, no antes: nadie te avisa al escribirlo.
- Por eso “un caso por variante” no es cuestión de estilo. Es lo que hace que la función esté **completa**.

Para la próxima

Lo que sigue

BNF y **struct**. Las figuras eran el ensayo: ahora las variantes van a ser las de un **lenguaje**, y el dato que armemos va a ser un programa.

Práctica

Hoja de ejercicios en Teams: definir los **struct** de un par de figuras más y extender **area** con **match**.

Se verifica sola en el REPL. **struct** y **match** son las dos herramientas que vamos a usar todos los días de aquí a diciembre.